

Pseudocode

Naive Bayes Algorithm

```
def classify( probs, instance)
  results = {}
  for label in class-labels
    results[ label] = probability_of( instance, label, probs)
  end
  results = normalize( results)
  best = find_best( results) # essentially argmax
  return (best, results)
end
```

Comments

This implementation does not show you how to calculate the probabilities. There are many different ways you could do that. You could iterate over the training examples and fill a Dict with counts and then either calculate the necessary probabilities immediately or on demand. Follow the programming assignment directions.

However, remember you should use +1 smoothing as discussed in the lectures. That is, every possible combination of attribute-values and class-labels should start with a virtual count of 1 instead of 0.

The true formula for Naive Bayes is:

$$label = \underset{c \in C}{argmax} \ p(c) \prod_{i,j} p(a_i = v_j | c)$$

However, it is often convenient to return the label with the highest probability as well as the entire probability distribution which is what the above pseudocode does.